Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:

lock(R1)

wait(100 ms)

lock(R2)

critical section

unlock(R2)

unlock(R1)

Process P2:

lock(R2)

wait(100 ms)

lock(R1)

critical section

unlock(R1)

unlock(R2)

Identify the exact condition that causes deadlock.

Suggest an optimisation to eliminate the deadlock without changing process logic.

Ans:

Given Pseudo Code

Process P1

lock(R1)

wait(100 ms)

lock(R2)

critical section

unlock(R2)

unlock(R1)

Process P2

lock(R2)

wait(100 ms)

lock(R1)

critical section

unlock(R1)

unlock(R2)

---

1. Exact Condition That Causes Deadlock

Execution Sequence

Step	Process P1	Process P2
1	Locks R1	
2		Locks R2
3	Waits 100 ms	Waits 100 ms
4	Requests R2 (already held by P2)	Requests R1 (already held by P1)
5	Waiting for R2	Waiting for R1

Now,

- P1 holds R1 and waits for R2.
- P2 holds R2 and waits for R1.

Neither process can continue.

This creates a Circular Wait, resulting in a deadlock.

Coffman Conditions Present

- ✓ Mutual Exclusion
- ✓ Hold and Wait
- ✓ No Preemption
- ✓ Circular Wait (main cause)

---

2. Optimization to Eliminate the Deadlock

Use a Consistent Lock Ordering

Both processes should acquire resources in the same order.

Modified Code

Process P1

lock(R1)

lock(R2)

critical section

unlock(R2)

unlock(R1)

Process P2

lock(R1)

lock(R2)

critical section

unlock(R2)

unlock(R1)

Why It Works

- Both processes request R1 first and R2 second.
- Circular waiting cannot occur because every process follows the same resource order.
- Deadlock is eliminated without changing the actual work performed by the processes.

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready.

You analyse the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready ...

Debug the cause using process management concepts and propose a system-level optimisation to

improve utilisation.

Ans:

Given

- CPU Utilization = 30%
- Processes are available in the Ready state.
- Scheduler log shows frequent transitions:

Ready → Waiting → Ready → Waiting → Ready ...

---

1. Debug the Cause

The repeated transition between Ready and Waiting indicates that the processes are I/O-bound.

Explanation

- A process is scheduled from the Ready queue.
- It executes briefly on the CPU.
- It quickly requests an I/O operation (disk, network, printer, etc.).
- The process moves to the Waiting state.
- After the I/O completes, it returns to the Ready state.

- This cycle repeats frequently.

Because processes spend most of their time waiting for I/O, the CPU remains idle much of the time, resulting in low CPU utilization (30%).

---

2. Process Management Concept

The issue is caused by:

- Frequent I/O waits
- I/O-bound processes
- Poor overlap of CPU and I/O activities

The scheduler has ready processes, but they do very little CPU work before blocking again.

---

3. System-Level Optimisation

Optimisation: Increase Multiprogramming

- Run more processes simultaneously.
- While one process waits for I/O, another ready process can use the CPU.
- This reduces CPU idle time and increases overall utilization.

Other possible improvements:

- Use asynchronous I/O or faster storage (e.g., SSDs) to reduce waiting time.
- Improve CPU scheduling (e.g., Round Robin or Multilevel Feedback Queue) to efficiently utilize CPU time.

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory

map shows:

● Code: 5 MB

● Heap: 150 MB

● Stack: 1 MB per thread

● 25 threads running

● Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

Ans:

Given

- **Code:** 5 MB
- **Heap:** 150 MB
- **Stack:** 1 MB per thread
- **Threads:** 25
- **Physical RAM:** 2 GB (2048 MB)

Memory Usage Calculation

- Code = **5 MB**
- Heap = **150 MB**
- Stack = **25 × 1 MB = 25 MB**

Total Memory Used = 5 + 150 + 25 = 180 MB

Available RAM = **2048 MB**

---

1. Identify the Memory Issue

Although the program uses only **180 MB**, which is much less than **2 GB of RAM**, it still experiences **frequent page faults**.

This indicates that the problem is **not insufficient physical memory**.

The likely causes are:

- Poor **memory access locality** (threads access scattered memory pages).
- **Heap fragmentation** or inefficient memory allocation.
- Multiple threads competing for memory, causing frequent page table lookups and cache/TLB misses.

These factors increase **page faults** and slow down execution.

---

2. Two Optimisations

Optimisation 1: Reduce Memory Fragmentation

- Use efficient memory allocation techniques such as **memory pools** or **object pooling**.
- Allocate memory in contiguous blocks to improve locality.
- This reduces unnecessary page faults.

Optimisation 2: Reduce Thread Stack Size or Number of Threads

- Lower the stack size if the application does not require 1 MB per thread.
- Use a **thread pool** instead of creating many threads.
- Fewer threads reduce memory overhead and improve cache efficiency.

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

● Q1 (Highest Priority): Interactive

● Q2: System

● Q3: Batch

● Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

P5 is in Q3 but keeps getting postponed.

Debug the root cause of starvation and suggest an optimisation to ensure fairness.

Ans:

Given

- Q1 (Highest Priority): Interactive, Time Quantum = 5 ms
- Q2: System, Time Quantum = 10 ms
- Q3: Batch, FCFS
- Process P5 is in Q3 but never gets CPU time.

---

1. Debug the Root Cause

The root cause is starvation.

Explanation

In a Multilevel Queue Scheduler:

- The scheduler always gives priority to Q1.
- If Q1 is empty, it schedules Q2.
- Q3 gets CPU time only when both Q1 and Q2 are empty.

Since interactive (Q1) and system (Q2) processes keep arriving, the CPU continuously serves them.

As a result:

- P5 remains in Q3 indefinitely.
- It never gets a chance to execute, causing starvation (indefinite postponement).

---

2. Optimization to Ensure Fairness

Aging Technique

Use Aging to gradually increase the priority of processes waiting for a long time.

Example:

- If P5 waits beyond a specified threshold, its priority is increased.

- Eventually, P5 is promoted from Q3 → Q2 or Q2 → Q1.
- Once promoted, it receives CPU time.

This ensures that no process waits forever.

Alternative Optimization

- Allocate a fixed percentage of CPU time (e.g., 10–20%) to lower-priority queues using time slicing between queues. This guarantees that Q3 processes receive CPU time even when higher-priority queues are busy.

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead.

Log details show:

● Average burst time per thread: 2ms

● I/O wait frequency: very high

● Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

Ans:

Given

- Context switches: 20,000 per second
- Average CPU burst time: 2 ms
- I/O wait frequency: Very high
- Scheduler: Preemptive
- Time quantum: 1 ms

---

1. Explain Why Context Switching Is So High

The high context-switch rate is caused by the combination of a very small time quantum (1 ms) and frequent I/O operations.

Reason

- Each thread requires about 2 ms of CPU time to complete its CPU burst.
- However, the scheduler allows only 1 ms before preempting the thread.
- The thread is switched out before completing its CPU burst.
- In addition, frequent I/O requests move threads between the Running, Waiting, and Ready states.
- Every preemption and I/O event results in a context switch.

As a result:

- CPU spends a significant amount of time saving and restoring process states.
- Less time is available for executing useful work.
- Overall system performance decreases due to high scheduling overhead.

---

2. Two Optimisations

Optimisation 1: Increase the Time Quantum

- Increase the quantum from 1 ms to a larger value (e.g., 4–8 ms).
- Threads can complete more of their CPU burst before being preempted.
- This reduces the number of unnecessary context switches and improves CPU efficiency.

Optimisation 2: Reduce I/O Waiting

- Use asynchronous I/O, I/O buffering, disk caching, or faster storage (SSDs).
- Threads spend less time entering the Waiting state.
- Fewer state transitions lead to fewer context switches and better throughput.